

Colab Guide

How to complete and submit your weekly assignments. This guide is linked from the syllabus, under **Colab guide**, and it is the only guide for this course.

For this course we use Google Colab to write and run code. Colab runs entirely in your web browser. You do not need to install anything on your computer. All you need is a Google account and an internet connection.

Key Vocabulary.

- A **cell** is one box in the notebook. Code cells hold code and show a `[ ]` marker on their left, which turns into a number once you run them. Text cells hold writing and have no marker. They are also called **markdown cells**, and they are where your written answers go.
- **Output** is whatever a code cell prints when you run it. This is typically a number, table, or plot. It appears directly underneath that cell.
- A **variable** is a named value your code has created, like `budget`; variables exist only while the notebook is running, which is why re-running matters.

Step 1 — Open the assignment

Click the assignment link on the syllabus. It opens the week's notebook directly in Google Colab. Make sure you are signed into a Google account first. (if you are signed into more than one account, pay attention to which account is shown in the top-right corner, because that is where your copy will be saved.)

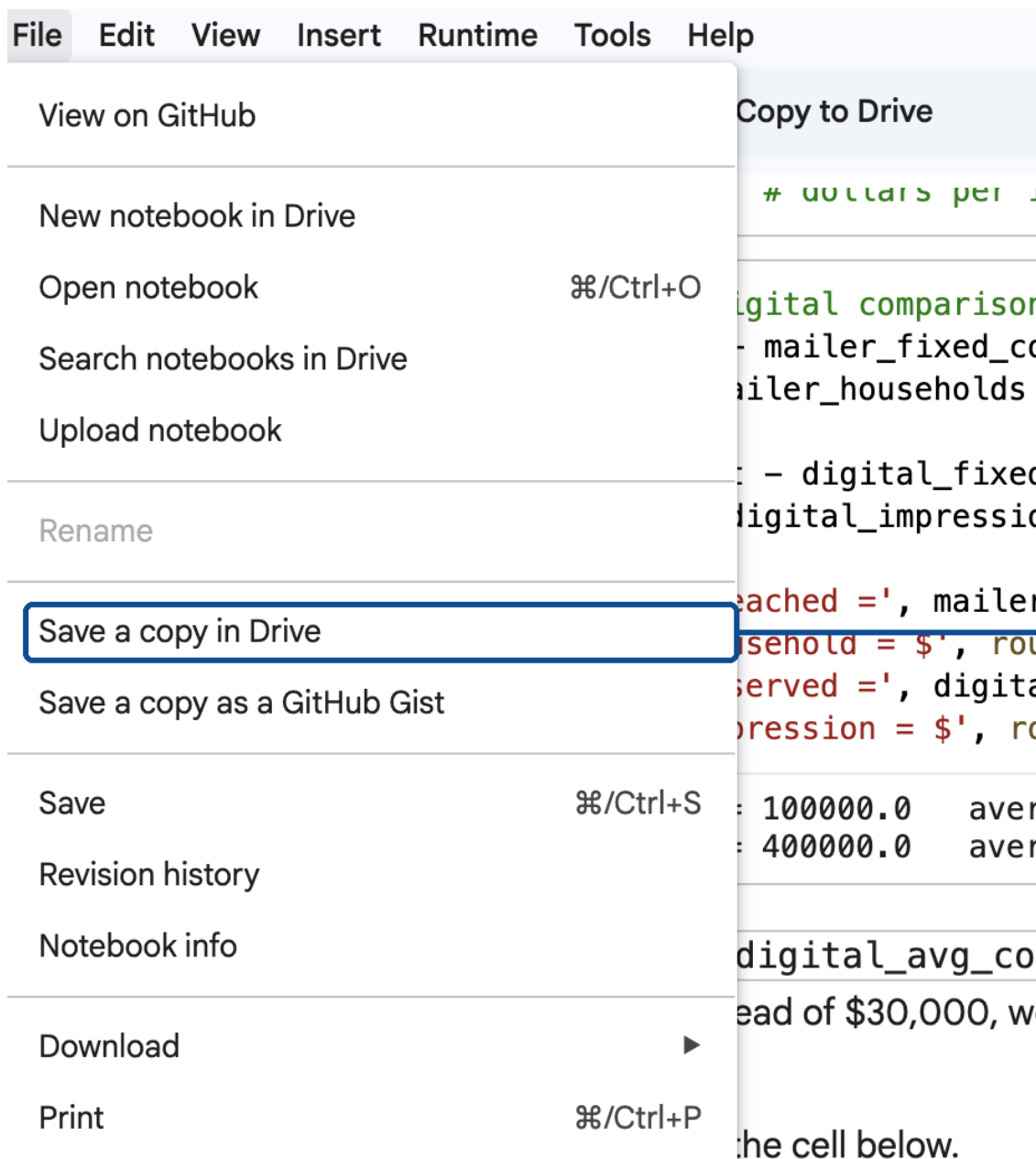
Step 2 — Save your own copy

The notebook that opens from the link is not yours. It looks and behaves like a normal notebook but **nothing you do in it is saved**. Instead of saving, Colab shows "Save in GitHub to keep changes" next to the filename, and when you close the tab your work is gone.

A screenshot of the Google Colab interface. On the left, the filename 'wk01_problem_set.ipynb' is displayed next to a circular refresh icon. To the right of the filename is a blue link with a cloud icon that says 'Save in GitHub to keep changes'.

If you ever see those words next to the filename, you are in the original, not your copy.

Before you do anything else, go to **File** → **Save a copy in Drive**. A new tab opens with your own copy. Work in that tab and close the original so you do not confuse the two. Your copy saves automatically to a folder called **Colab Notebooks** in your Google Drive.



Three items in that menu look similar. **Save a copy in Drive** is the one you want. Not **Save a copy as a GitHub Gist**, directly below it. Not **Save**, which does nothing here. There is also a **Copy to Drive** button in the toolbar that does the same job as Save a copy in Drive. Either is fine.

If you find yourself retyping work you already did, this is almost always the step you skipped.

Step 3 — Run the setup cell

The setup cell is the first code cell in the notebook, under the heading Setup. Run it before anything else, and do not edit it.

From Week 2 on, the notebook also loads the week's dataset for you, straight from the course repository. Either way there is nothing to download and nothing to upload; you just need to be online.

When you run a cell you may see a warning: *"This notebook was not authored by Google."* Click **Run anyway**. It is expected for every notebook in this course, and you will see it again each time the notebook restarts.

Warning: This notebook was not authored by Google

This notebook is being loaded from [GitHub](#). It may request access to your data stored with Google, or read data and credentials from other sessions. Please review the source code before executing this notebook.

Cancel

Run anyway

The warning appears because the notebook comes from the course's GitHub repository rather than from Google. Course notebooks only read a public data file; they cannot reach anything in your Drive or your account.

Step 4 — Do the work

To run a cell: click anywhere inside it, then press **Shift+Enter** (hold Shift, press Enter). Shift+Enter runs the cell and moves you down to the next one. You can also hover over a cell and click the ► button that appears at its left edge.

```
[ ] # Reproduce the mailer and digital comparisons from class.
    mailer_households = (budget - mailer_fixed_cost) / mailer_marginal_cost
    mailer_avg_cost = budget / mailer_households

    digital_impressions = (budget - digital_fixed_cost) / digital_marginal_cost
```

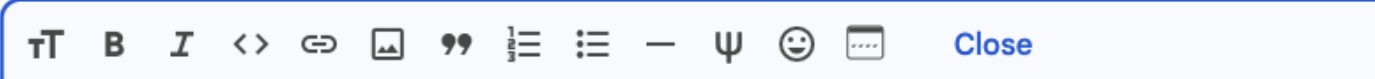
RUN BUTTON

Cells you have to change are usually marked one of two ways:

- A comment beginning `# TODO`: a working cell where you edit one value and re-run it.
- A comment containing `# YOUR CODE HERE`: an otherwise empty cell where you write the code yourself. Some weeks add a note after it saying what to compute.

Written answers go in markdown cells. Double-click one to open it. Your raw text appears on the left and a live preview on the right; replace the placeholder text with your answer, then press **Shift+Enter** or click **Close** to render it and shut the editor.

WHAT YOU TYPE (left half of the cell)



****Your answer:****

My answer goes here. I can write as many sentences as I need, and ****bold text**** looks like this once I press Shift+Enter.

HOW IT WILL LOOK (right half)

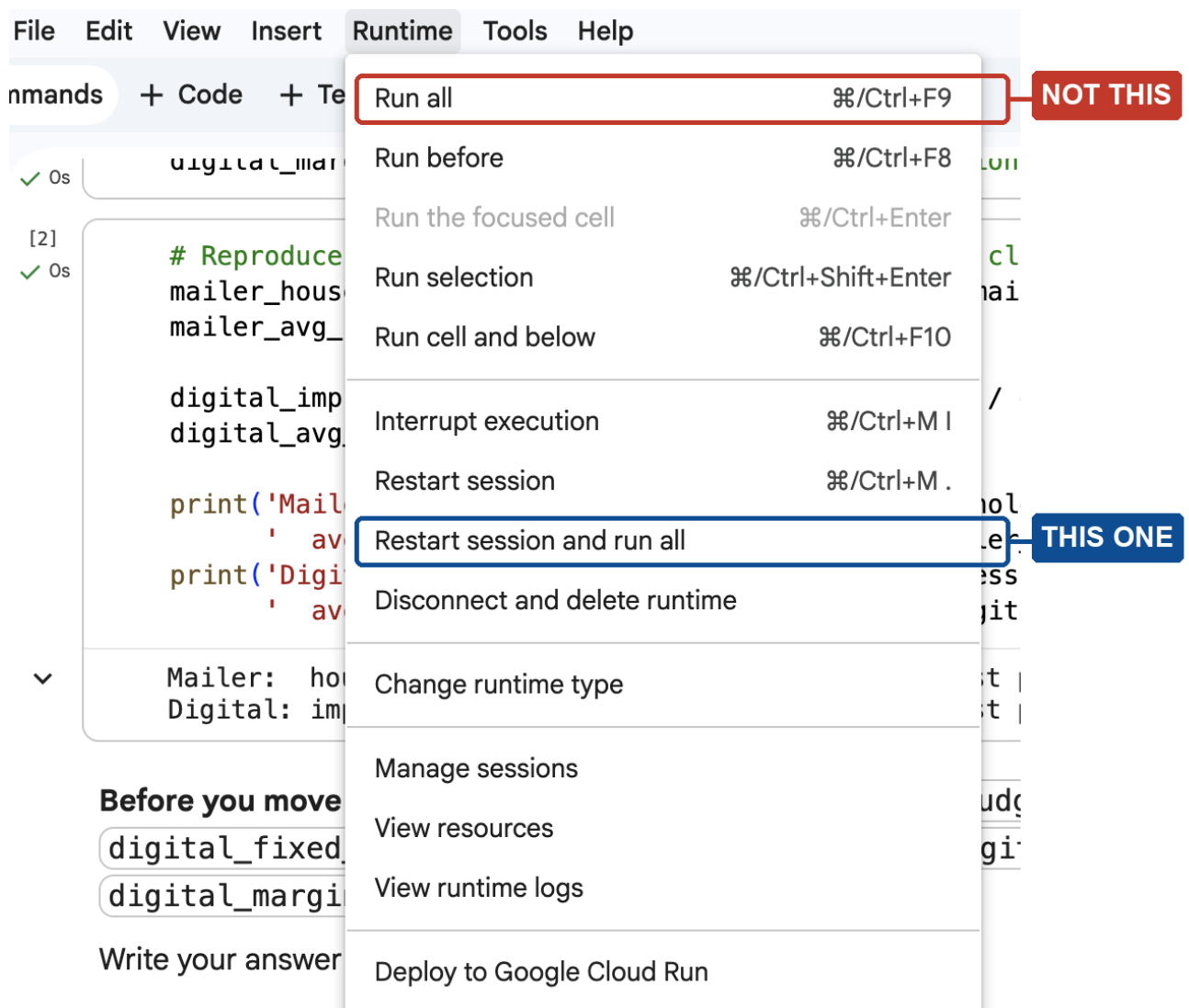
Your answer:

My answer goes here. I can write as many sentences as I need, and **bold text** looks like this once I press Shift+Enter.

Step 5 — Submit

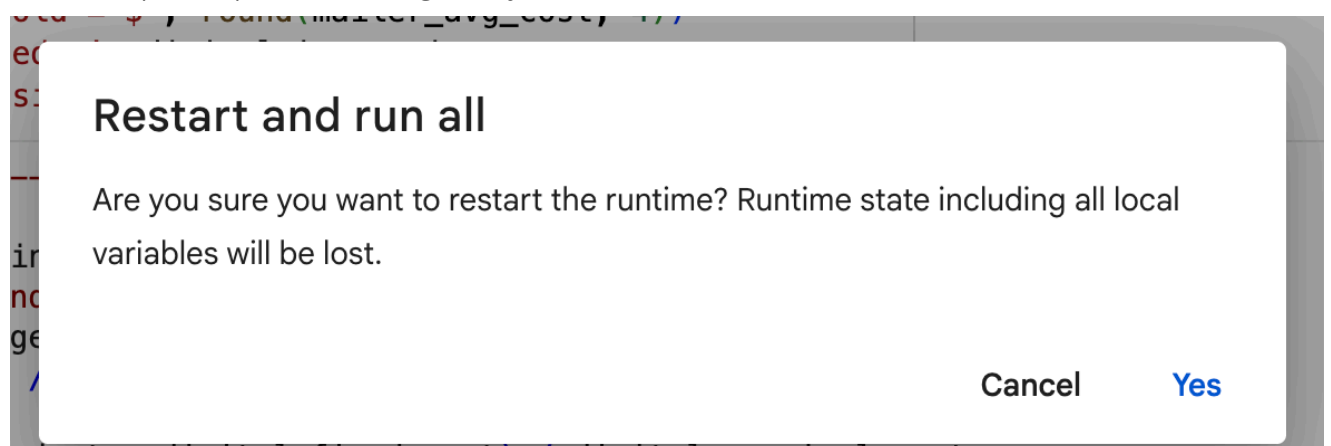
Do these six things in order.

1. **Runtime** → **Restart session and run all**. Do this only after you have finished every task and written your answers. It clears every variable and re-runs the notebook top to bottom, in order, which proves the version you are handing in actually works start to finish. Code that only worked because you ran cells in an unusual order will break here.



Not "Run all," which sits at the top of the same menu and is the easy mis-click. Run all keeps the variables you already have, so a notebook that only works out of order still looks perfect.

Colab then asks you to confirm, in a box titled **Restart and run all** (a shorter name for the same thing), warning that "all local variables will be lost." Click **Yes**. Your code and your written answers are saved; only the *results* are cleared, and this step recomputes them straight away.



2. **Wait for it to finish, then check that every cell ran.** The run takes a few seconds and works down the notebook one cell at a time. While it is going, the top-right corner reads **Restarting** or shows a spinner; it is finished when that goes back to **Connected** with a green check.

... Restarting

STILL RUNNING

✓ Connected

FINISHED

The surest check is the bottom of the notebook: **scroll to the very last code cell and confirm it has a number**. Until then, cells lower down still show `[ ]` because they are queued and have not had their turn.

Once it is done, scroll from the top. A cell that ran shows a number and a green check in its left margin, and its output underneath. (The `0s` beside the check is just how long the cell took to run.)

[2]

✓ 0s

```
# Reproduce the mailer and digital comparisons from class.
mailer_households = (budget - mailer_fixed_cost) / mailer_marginal_cost
mailer_avg_cost = budget / mailer_households

digital_impressions = (budget - digital_fixed_cost) / digital_marginal_cost
digital_avg_cost = budget / digital_impressions

print('Mailer: households reached =', mailer_households,
      ' average cost per household = $', round(mailer_avg_cost, 4))
print('Digital: impressions served =', digital_impressions,
      ' average cost per impression = $', round(digital_avg_cost, 4))
```

▼

Mailer: households reached = 100000.0 average cost per household = \$ 0.5

Digital: impressions served = 400000.0 average cost per impression = \$ 0.125

[]

```
# Reproduce the mailer and digital comparisons from class.
mailer_households = (budget - mailer_fixed_cost) / mailer_marginal_cost
mailer_avg_cost = budget / mailer_households

digital_impressions = (budget - digital_fixed_cost) / digital_marginal_cost
digital_avg_cost = budget / digital_impressions

print('Mailer: households reached =', mailer_households,
      ' average cost per household = $', round(mailer_avg_cost, 4))
print('Digital: impressions served =', digital_impressions,
      ' average cost per impression = $', round(digital_avg_cost, 4))
```

That is the same cell before it was run: empty brackets, no check, no output. In the PDF it will look like a missing answer, because it is one.

A green check is not proof you answered. An untouched `# YOUR CODE HERE` cell is just a comment, so it runs perfectly and prints nothing at all. Below, Task 3 asked for two printed numbers and got none, and nothing on screen complains:

Check: you should get 2,100 conversations and an average cost of about \$23.81.

[5]

✓ 0s

```
# YOUR CODE HERE
```

So check for **output**, not for the absence of red. Every task cell should have printed something underneath it. If one hasn't, you either left it blank or deleted your attempt.

If a cell defeats you, that is different from not trying. Leave your best attempt in the cell, add a text cell underneath saying where you got stuck, and submit. Do not delete the task. Problem sets are graded on whether you attempted everything, so a notebook that stops at a cell you genuinely could not solve still counts. A blank you never touched does not.

3. File → Print.

Save ⌘/Ctrl+S
Revision history
Notebook info
Download ▶
Print ⌘/Ctrl+P

100000.0 aver

400000.0 aver

digital_avg_cost

head of \$30,000, we

THIS ONE

the cell below.

- In the print window that opens, choose "Save as PDF," then click Save. In Chrome, look for the **Destination** dropdown near the top right and set it to **Save as PDF** rather than a printer. (If you end up in your Mac's own print dialog instead, the equivalent is the **PDF** menu at the bottom left.)
- Open the PDF and read it before you upload.** The PDF prints every heading and prompt whether or not your code ran, so **it will look complete even when it isn't**. What matters is the output: under each code cell there should be a number, a table, or a plot. A red error box, or a `[ ]` with nothing beneath it, means that part did not run and will be graded as missing.

[6] 0s

YOUR CODE HERE

canvass_marginal_cost = 20

canvass_conversations = (budget - canvass_fixed_cost) / canvass_marginal_cost

NameError

Traceback (most recent call last)

/tmp/ipykernel_713/2341964870.py in <cell line: 0>()

1 # YOUR CODE HERE

2 canvass_marginal_cost = 20

----> 3 canvass_conversations = (budget - canvass_fixed_cost) / canvass_marginal_cost

NameError: name 'canvass_fixed_cost' is not defined

That is what a failed cell looks like: a red marker beside the cell number and a *traceback* (Python's report of what went wrong) instead of a result. The useful part is the last line, here `NameError: name 'canvass_fixed_cost' is not defined`. When a cell fails during restart-and-run-all, every cell after it is skipped, so a single red box near the top can empty out the whole rest of your PDF.

Also confirm your written answers printed in full, and that no plot is cut off at a page break.

If a long line of code runs off the right edge, split it across two lines and print again. Colab prints code without wrapping it, so anything past the edge of the page is simply cut. **See *Splitting a Long Line of Code at the end of this guide before you try it***, because breaking a line in the wrong place stops your code from running. If you cannot get it to work, put the line back the way it was and submit: a cut-off line does not cost you credit.

- Upload the PDF to Canvas.** You only need to submit the PDF to Canvas.

You do not need to put your name on the notebook. Colab will call your copy something like "Copy of wk01_problem_set" and that is fine. Canvas records who submitted, so your work is already attached to you.

Troubleshooting

What you see	What it means and what to do
Your work vanished, or the filename says "Save in GitHub to keep changes"	You are typing in the original notebook, not your copy — it looks editable but saves nothing. File → Save a copy in Drive , then work in the new tab. Past copies are in your Google Drive, under <i>Colab Notebooks</i> .
"Warning: This notebook was not authored by Google"	Expected. Every notebook in this course comes from the course GitHub repository. Click Run anyway .
A cell has a number and a green check but printed nothing	Either it is a setup cell, which prints nothing by design, or it is a blank you never filled in — an empty <code># YOUR CODE HERE</code> cell runs without complaining. Check that every task cell printed something.
<code>NameError: name 'something' is not defined</code>	You ran cells out of order, or the session restarted and wiped your variables, or you never created that variable. Your code has to run in order, top to bottom. Runtime → Restart session and run all .
<code>SyntaxError: invalid syntax</code> pointing at a blank you were meant to fill in	In some weeks the blank is written so the notebook refuses to run until you fill it in. That is by design. Write your code on that line.
<code>HTTPError</code> , <code>URLError</code> , or <code>ConnectionError</code> on a cell that loads data	Either you are offline or that cell was edited. Check your connection; if you changed the cell, re-open a fresh copy from the syllabus link.
<code>FileNotFoundError</code> mentioning a <code>.csv</code>	You are running an old copy of the assignment. Open the current notebook from the syllabus link.
"Runtime disconnected" or everything suddenly undefined	The <i>runtime</i> is the machine in the cloud running your code — Colab calls it a <i>runtime</i> in some menus and a <i>session</i> in others, and they are the same thing. Colab shuts idle ones down, taking every variable with them. Reconnect, then restart-and-run-all. Your <i>code</i> is saved; only the <i>results</i> are lost.
The PDF looks complete but some outputs are missing	The run stopped at an error partway down, so every cell after it printed its code but no result. Find the red error box, fix that cell, restart-and-run-all, and export again.
A line of code runs off the right edge of the page	Colab does not wrap code when it prints, so anything past roughly 70 characters is cut. Split the line so all of it shows, following <i>Splitting a Long Line of Code</i> below. Do not just press Enter in the middle of the line ; that stops the code running.
A long output or plot is cut off in the PDF	A long output can sit in a scroll box — right-click it → Disable scrolling for outputs . A collapsed section does not print, so click the ^ arrows to expand every section before printing.
A plot is missing from the PDF	That cell did not run. Restart-and-run-all, confirm the plot appears on screen, then print.
<code>ModuleNotFoundError</code>	Rare in Colab. Add a new cell, run <code>!pip install statsmodels</code> , then restart-and-run-all.
A cell runs forever	Click the stop button, then Runtime → Restart session and run all . If it hangs again on the same cell, bring it to office hours.

If none of this fixes it, come to office hours or email for assistance. Include a screenshot of the full error message and your code — the last line of a Python error is the useful part.

Splitting a Long Line of Code

When a line of code runs off the right edge of the printed page, split it so that all of it shows. This section is how to do that without breaking your code. Read it before you start editing.

Breaking a line in the wrong place stops your code from running. Python treats the end of a line as the end of an instruction, so you cannot just press Enter in the middle of one.

The safe rule: break inside brackets that are already there, either right after a comma or just before a `+`, `-`, `*` or `/`. Round `()`, square `[]` and curly `{ }` brackets all count. Python knows the instruction is not finished until the bracket closes, so it ignores the line break. Never break inside quote marks, and never in the middle of a word or a number.

Too long:

```
print('turnout', round(gap, 2), 'percentage points')
```

Split, and still works:

```
print('turnout', round(gap, 2),  
      'percentage points')
```

If there are no brackets to break inside, add a pair around the right-hand side first. **The opening bracket has to stay up on the first line.**

Too long:

```
total = price * quantity + shipping
```

Split, and still works:

```
total = (price * quantity  
        + shipping)
```

Splitting a long piece of text

You cannot break inside quote marks. What you can do is close the quote, open a new one on the next line, and let Python join the pieces. **This only works inside brackets.** It is how every `read_csv` line in these notebooks is written:

```
df = pd.read_csv(  
    'https://raw.githubusercontent.com/joshuakalla/'  
    'data_science_campaigns_26/main/weeks/'  
    'wk03_rcts_and_ate/data/gotv_2014.csv')
```

Two things to watch. **Keep the spaces:** `'percentage ' 'points'` gives "percentage points", but `'percentage' 'points'` gives "percentagepoints". And **stay inside the brackets.** Outside them the second piece is thrown away silently:

```
x = 'percentage '  
    'points'
```

That runs with no error at all and leaves `x` as just "percentage ".

Run the cell again before you print. Most wrong breaks show up as a red error box straight away. The string case just above is the one that does not, so if you split text, read the output and check it still says what you meant. Cmd-Z (Ctrl-Z on Windows) undoes the change.